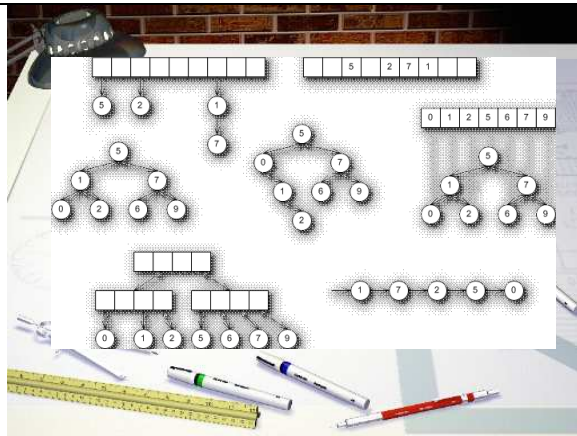




Slide 1

CS 170

Java Programming 1

ArrayLists and Generic Collections

Duration: 00:01:52

Advance mode: Auto

Notes:

Hi Folks. Welcome to the CS 170, Java Programming 1 lecture on ArrayLists and Generic Collections.

In addition to arrays, the Java Class Libraries include many built-in classes designed to store data elements and to manipulate them in different ways. The general term for these kinds of classes are data structures. If you're a Computer Science major, you'll even take a course devoted entirely to these data structures. In Java, these data structure classes are known as the Java Collection classes.

Before the Java Collection classes were introduced, Java had a set of rudimentary data structures designed to store generic Object references. The most commonly used of these was the Vector class. When you placed an item in a Vector, the Vector itself didn't keep track of what "kind" of thing it was. Instead, it was up to you as a programmer to cast each element to the actual correct type after retrieving it from the collection, but before using it. Because this lead to many different kinds of runtime errors, these earlier collection classes are known as "unsafe" collections. Today, the javac compiler will usually warn you not to use them.

Java 5 introduced a new breed of data structures that allow you to specify the type of element stored in a collection, by using a type parameter, a facility that is known as generics. The generic collection class that's most similar to the regular array class is called the ArrayList, and that's what we'll look at in this lecture.

Remember, though, that if you compile your code using the new generic classes, your code won't run on pre-Java 5 Virtual Machines. This is a Java 5 and up feature.

Introducing ArrayLists

- **ResizableArray** solves the **fixed size** problem
 - Class still bound to a **specific type**, though
 - Need a **different** class for each kind of element
- **Generic collections** introduced in Java 5 change this
- In Java, an **ArrayList** is a "growable" array
 - It also can hold any kind of object-based element
 - Can **only** hold objects, not primitives (boxing, etc.)
 - Can be a little less efficient than arrays
 - Must use methods to retrieve and store objects

Slide 2

Introducing ArrayLists

Duration: 00:02:21

Advance mode: Auto

Notes:

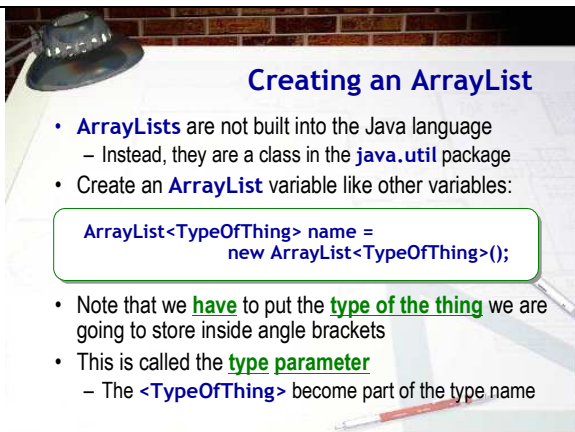
As you saw in that last lecture, one of the problems with using arrays, is that they are fixed-size. If you plan for ten elements when you create your array, you can't change your mind after the program is running, and add an eleventh element. You can get around that, though, by creating your own class, like the ResizableArrayClass from the last lesson, that hides those details with a private internal array and then writing code to reallocate a new array and copy the old elements when the original array elements are filled.

The ResizableArrayClass still has a few problems, though, because you have to create a separate ResizableArrayClass for each kind of element you want to store, unlike regular arrays, where you can store any kind of object or primitive type. Fortunately, the new Java 5 collection classes change this.

The Java "growable" array class is called the ArrayList. With the ArrayList:

- you can store any kind of object-based element. You can have an ArrayList of Strings, Cards, Integers, and so on.
- while you can store any kind of object-based element, you can't store primitive values, like you can with arrays. If you want to create an ArrayList of int, you have to use the Integer wrapper class type instead. Then, when you store and retrieve int values in the collection, they are automatically converted to and from the wrapper type, in a process called auto boxing.
- because of this "boxing" using ArrayLists to store primitive types is much less efficient than using arrays. Even for object types, using ArrayList may be a little slower than the equivalent array class.
- also, you may find the ArrayList a little less convenient because instead of using bracketed subscripts to store and retrieve elements, you have to use a set() or get() method instead.

Usually, though, the small loss in efficiency and convenience is more than made up by the greater convenience of auto-sizing and by the large number of built-in methods that can considerably reduce the amount of code you need to write.



Creating an ArrayList

- **ArrayLists** are not built into the Java language
 - Instead, they are a class in the **java.util** package
- Create an **ArrayList** variable like other variables:

```
ArrayList<TypeOfThing> name =
    new ArrayList<TypeOfThing>();
```

- Note that we **have** to put the **type of the thing** we are going to store inside angle brackets
- This is called the **type parameter**
 - The **<TypeOfThing>** become part of the type name

Slide 3

Creating an ArrayList

Duration: 00:01:24

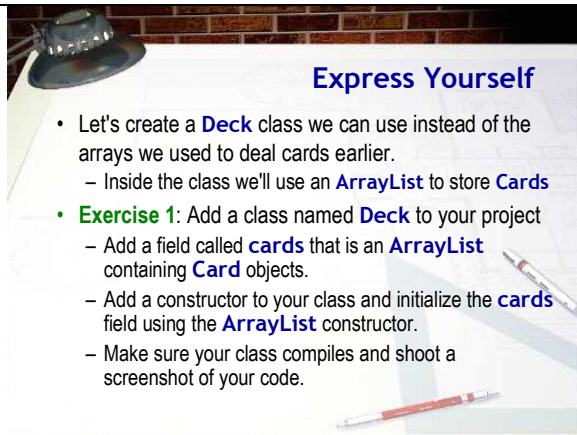
Advance mode: Auto

Notes:

The ArrayList class is not built in to the Java language like the array type is. Also, the ArrayList is not part of the automatically included java.lang package, like the String and Math classes are. The ArrayList is part of the java.util package, so to use them, you'll need to import java.util.ArrayList or java.util.*;

You create an ArrayList using syntax that is similar to the syntax for creating other object variables and then calling an object constructor. Since ArrayLists are automatically sized, you don't need to specify their size, like you do with an array constructor.

The biggest difference between a regular object constructor and a generic type constructor, like ArrayList, is the ArrayList type parameter, which must appear right after the name ArrayList, enclosed in angle brackets, kind of like an HTML tag. When you supply a type parameter like this, it actually defines an entirely new class, such as ArrayList<Integer> or ArrayList<Double>. The type parameter actually becomes part of the class name, used both when you declare the variable and when you invoke the constructor.



Express Yourself

- Let's create a **Deck** class we can use instead of the arrays we used to deal cards earlier.
 - Inside the class we'll use an **ArrayList** to store **Cards**
- Exercise 1:** Add a class named **Deck** to your project
 - Add a field called **cards** that is an **ArrayList** containing **Card** objects.
 - Add a constructor to your class and initialize the **cards** field using the **ArrayList** constructor.
 - Make sure your class compiles and shoot a screenshot of your code.

Slide 4 

Express Yourself

Duration: 00:01:30

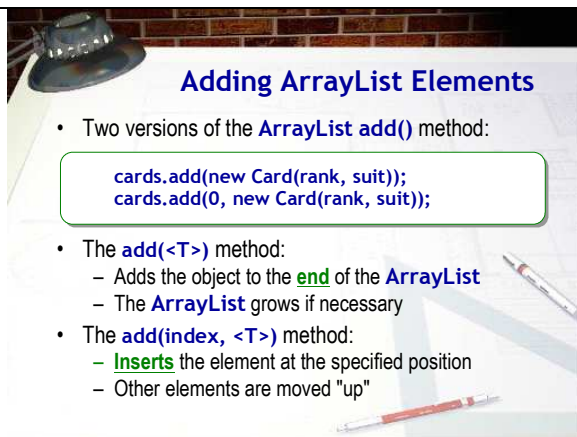
Advance mode: Auto

Notes:

Let's put the ArrayList to work. In earlier lectures, I showed you how to use an array to simulate a deck of cards. Let's create a Deck class and use an internal ArrayList to store the Card objects. We'll continue working with the ic19 project. Here's what you should do.

- Using Eclipse, add a new class named Deck to your project. Make sure you click the checkbox that says generate comments when you complete the New Class dialog.
- Inside the class, add an instance variable named cards that is an ArrayList containing Card objects. Just create the instance variable itself; don't initialize it. If Eclipse shows you an error at this point, you can use the Quick-Fix feature to add the necessary import.
- Now, write a constructor for your class. It doesn't have to take any parameters. Inside the constructor, initialize the field named cards using the ArrayList constructor.

If you've done everything right, your class should compile with no little red squiggles. Shoot me a screenshot of your code so I can see that's the case. Then, start a new section in your IC document and paste it in as Exercise 1.



Adding ArrayList Elements

- Two versions of the **ArrayList add()** method:


```
cards.add(new Card(rank, suit));
cards.add(0, new Card(rank, suit));
```
- The **add(<T>)** method:
 - Adds the object to the **end** of the **ArrayList**
 - The **ArrayList** grows if necessary
- The **add(index, <T>)** method:
 - Inserts** the element at the specified position
 - Other elements are moved "up"

Slide 5 

Adding ArrayList Elements

Duration: 00:01:37

Advance mode: Auto

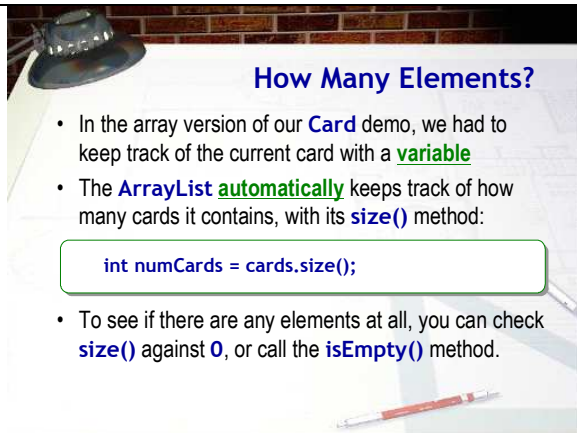
Notes:

When you first create an ArrayList, it has the potential for holding then elements. This is known as the ArrayList's capacity. However, it doesn't actually contain any elements; instead, it is completely empty. This is known as the ArrayList's size. With arrays, size and capacity are identical.

Instead of using subscripts to place element in the predefined "slots", like you do with an array, you actually add the element to the array, which creates the appropriate slot and places the element into it. There are actually two versions of the add() method.

The first version shown here, called add(<T>), adds the element to the end of the ArrayList, after the last element. If you've exhausted the capacity of the ArrayList, it's automatically expanded as necessary.

The second version of add, add(index, <T>), really should be named insert, since it places an element at a particular index position. In my second example here, I'm placing the new Card in the first position, position 0. With this insert version of add(), the element that was originally in position 0 (in this case) is moved to position 1, the element in position 1 is moved to position 2 and so on. As with the original version of add(), the ArrayList capacity is expanded when you insert additional elements.



How Many Elements?

- In the array version of our **Card** demo, we had to keep track of the current card with a **variable**
- The **ArrayList** **automatically** keeps track of how many cards it contains, with its **size()** method:


```
int numCards = cards.size();
```
- To see if there are any elements at all, you can check **size()** against **0**, or call the **isEmpty()** method.

Slide 6

How Many Elements?

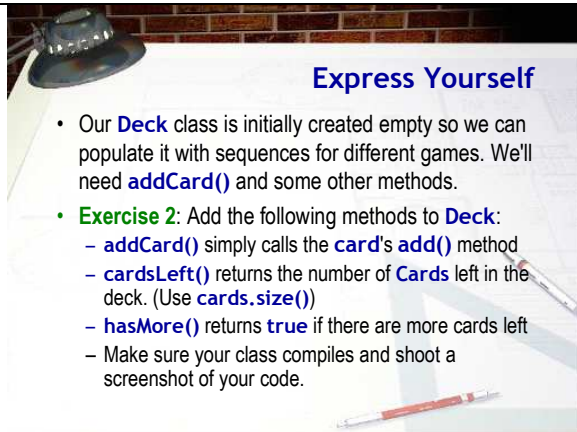
Duration: 00:00:41

Advance mode: Auto

Notes:

In the partially-filled array version of the CardDemo program, from last lecture, we had to keep track of the current card by using a variable. We don't have to do that with the ArrayList class, though, because the class automatically keeps track of how many elements the list contains. You can retrieve that information by calling the size() method like this.

If we want to find out if the deck is empty, we can use size(). If cards.size() is 0, then the deck is empty. The ArrayList also has an isEmpty() method that does the same thing.



Express Yourself

- Our **Deck** class is initially created empty so we can populate it with sequences for different games. We'll need **addCard()** and some other methods.
- Exercise 2:** Add the following methods to **Deck**:
 - addCard()** simply calls the **card's add()** method
 - cardsLeft()** returns the number of **Cards** left in the deck. (Use **cards.size()**)
 - hasMore()** returns **true** if there are more cards left
 - Make sure your class compiles and shoot a screenshot of your code.

Slide 7

Express Yourself

Duration: 00:01:21

Advance mode: Auto

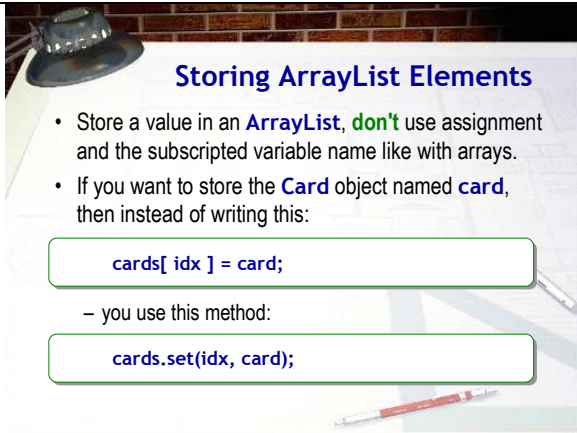
Notes:

Rather than filling in the cards ArrayList in our Deck constructor, we'll leave it empty so that we can use the same class for several different games. After all, not all games require just the 52 cards of the standard deck.

To allow the user of the Deck class to populate it, we'll need to supply an addCard() method, along with a couple other methods that make working with the Deck class a little easier. Here are the methods you should add to Deck:

- the addCard() method takes a single playing Card as it's parameter and then uses the ArrayList add() method to place it at the end of the cards ArrayList.
- the cardsLeft() method returns the number of Cards left in the deck. You can use the ArrayList size() method to determine this value.
- the hasMore() method returns true if there are more cards left. You can use the cardsLeft() method to generate the return value for this method.

Once you've written these three methods, make sure your class compiles (no squiggles) and shoot me a screen-shot. Paste it into your IC document as Exercise 2.



Storing ArrayList Elements

- Store a value in an **ArrayList**, **don't** use assignment and the subscripted variable name like with arrays.
- If you want to store the **Card** object named **card**, then instead of writing this:

```
cards[ idx ] = card;
```

– you use this method:

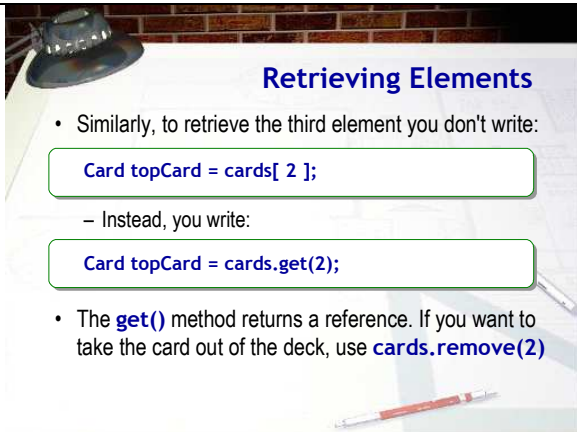
```
cards.set(idx, card);
```

Slide 8 🎧
Storing ArrayList Elements
 Duration: 00:00:58
 Advance mode: Auto

Notes:

With arrays, you use a subscripted array name on the left-hand-side of an assignment expression to place a value into a specific slot in the array. With an ArrayList, though, you have to call a method instead. The easiest way to see how this works is to compare an array assignment to the equivalent operation with an ArrayList.

If you want to store a Card object named card in the cards collection, instead of writing `cards[idx] = card;` like you would when using an array, use the `set()` method, like this: `cards.set(idx, card)`. Note that the relative positions of the words cards, idx, and card are the same in both cases.



Retrieving Elements

- Similarly, to retrieve the third element you don't write:

```
Card topCard = cards[ 2 ];
```

– Instead, you write:

```
Card topCard = cards.get(2);
```

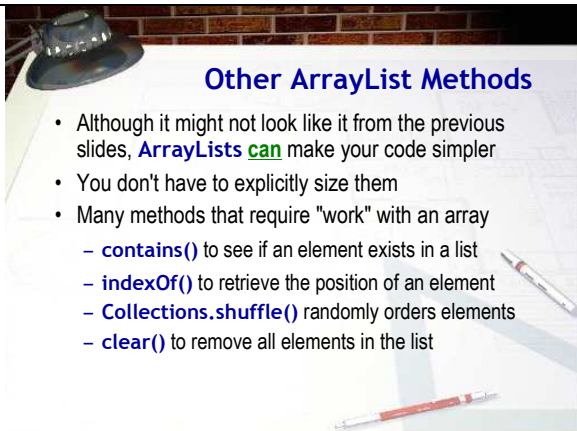
- The **get()** method returns a reference. If you want to take the card out of the deck, use **cards.remove(2)**

Slide 9 🎧
Retrieving Elements
 Duration: 00:00:39
 Advance mode: Auto

Notes:

Similarly, to retrieve an element, instead of using a subscripted array name on the right, like this: `card topCard = cards[2];` you use the `get()` method, passing in the appropriate index value like this: `Card topCard = cards.get(2);`

The `get()` method does not remove the specific element from the ArrayList, but simply returns a reference. If you want to remove the element from the list after retrieving it, you need to use the ArrayList `remove()` method like this.



Other ArrayList Methods

- Although it might not look like it from the previous slides, **ArrayLists can** make your code simpler
- You don't have to explicitly size them
- Many methods that require "work" with an array
 - contains()** to see if an element exists in a list
 - indexOf()** to retrieve the position of an element
 - Collections.shuffle()** randomly orders elements
 - clear()** to remove all elements in the list

Slide 10 🎧
Other ArrayList Methods
 Duration: 00:01:36
 Advance mode: Auto

Notes:

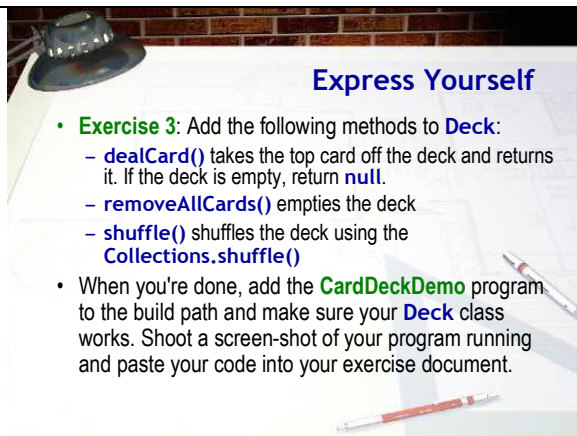
Although storing and retrieving objects in an ArrayList is more work than using an array, many common operations that you have to perform "by hand" when using arrays are automatic when using ArrayLists. That means that using ArrayLists can actually make your code much simpler, rather than more complex.

For instance:

- Most importantly, you don't have to explicitly size an ArrayList. This saves you both memory (because you don't need to always include unneeded elements just in case) and effort.
- To find out if a particular item is contained in an ArrayList, you don't need to use a loop; you can just use the `contains()` method.
- If you already know that a particular object is located in an ArrayList, you can retrieve its position using `indexOf()`.

- You can remove an object from a `ArrayList` using `remove()`. When you do this, the other objects move "down" to close up the gap left by the departee. Likewise, as you've already seen, you can insert an element at any position, and those elements already in place will move over to make room for the newcomer.
- To randomly order the elements in an `ArrayList`, you can use the static `shuffle()` method from the `Collections` class.
- Finally, you can remove all of the elements from an `ArrayList` with the `clear()` method.

When you wrote your own array classes, these were all things that you had to do on your own by writing various loops and comparisons.



Express Yourself

- **Exercise 3:** Add the following methods to **Deck**:
 - **dealCard()** takes the top card off the deck and returns it. If the deck is empty, return **null**.
 - **removeAllCards()** empties the deck
 - **shuffle()** shuffles the deck using the **Collections.shuffle()**
- When you're done, add the **CardDeckDemo** program to the build path and make sure your **Deck** class works. Shoot a screen-shot of your program running and paste your code into your exercise document.

Slide 11 
Express Yourself
 Duration: 00:01:55
 Advance mode: Auto

Notes:

OK, your turn again. Let's write a few more methods for our Deck class.

- The `dealCard()` method takes the top card off the deck and returns it. You'll use the `get()` method to get a reference to the top card, and the `remove()` method to remove it from the `ArrayList`. If the cards list is empty, then the `dealCard()` method should return `null`.
- The `removeAllCards()` method empties the deck. You can do this by using a single `ArrayList` method.
- The `shuffle()` method shuffles the deck. You can do this by simply calling `Collections.shuffle()` and passing the cards `ArrayList` instance variable as a parameter.

When your done, locate `CardDeckDemo.java` in the `ic19` project. Since this class requires a completed `Deck` class to compile, it's not included with the other Java source files. Instead, you'll find it down near the bottom of the project. To include it, right-click and select `Build-Path->Include`. When you do this, it will "jump" up to join the other Java classes.

For Exercise 3, run `CardDeckDemo` and shoot me a screenshot. Below your screenshot, copy and paste the completed code from your `Deck` class.

Well, that's it for the `ArrayList` class. Before we leave this lecture, though, I want to briefly mention two other array topics that really aren't large enough to warrant having a lecture of their own: command-line arguments and variable-length parameters. Both of these are covered in Chapter 7.

Command Line Arguments

- When you want to pass arguments to **applications**
 - Like when you need to compile your code
- Exercise 4:** Create a console app **Args**, and try it
 - Loop through the **args** array, using the **length** field
 - Print each element to the console
 - Run using `java Args one two "three four"`

```
C:\WINDOWS\System32\cmd.exe
C:\docs\cs170\F05\ec1ipse\unit08>java Args One Two "Three Four"
One
Two
Three Four
C:\docs\cs170\F05\ec1ipse\unit08>
```

Slide 12

Command Line Arguments

Duration: 00:02:29

Advance mode: Auto

Notes:

One of the most common uses of String arrays is when you create an application and you need to pass information into that application when you run it. With methods, you pass information into the method by supply actual arguments which are placed in the method's formal parameters. With applications, information is passed to your program via the command line, similar to the information you supply when you use `javac` to compile your Java code from the terminal window.

When you manually compile your code, you type in the word `javac`, and then follow that with the name of the program you want to compile, like this: The program file name you supply, `VoteCounter.java` in the screenshot shown here, is an example of a command line argument.

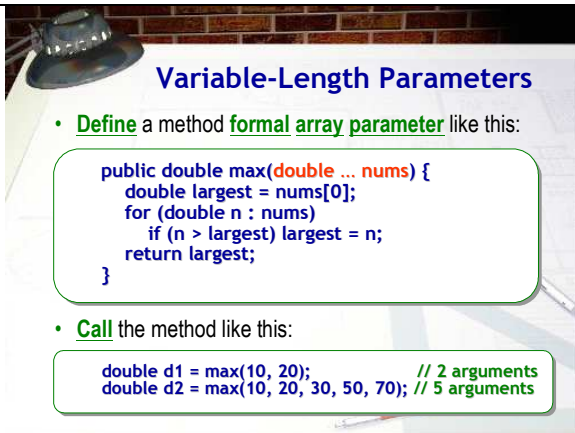
As you'll recall, when a Java application begins, it starts with the `main()` method. The `main()` method is passed a single parameter from the operating system, an array of String, traditionally called `args`. (Of course, you can name it anything you like.)

Inside your `main()` method, you can check the field `args.length` to see how many elements the `args` array contains. If no arguments were passed to your application, then `args.length` will be 0.

Since `args.length` tells you how many command-line arguments there are, the situation practically begs for a loop, right? Let's create an class that just uses a loop to print out the command-line arguments.

- Create a console-mode application named `Args`. Make sure you include a `main()` method.
- Inside the `main()` method, loop through the array (`args`) and print each command line argument to the console.

Once you have the program finished, open a command window run it from the command line, using `java Args one two "three four"`. Here's what the program should look like as it runs. Notice that the word "One" is `args[0]`, "Two" is `args[1]`, and the quoted String "Three Four" are placed together in `args[2]`.

The slide features a background image of a desk lamp and a brick wall. The title "Variable-Length Parameters" is in blue. A green bullet point states: "Define a method **formal array parameter** like this:". Below this, a code block shows a Java method:

```
public double max(double ... nums) {
    double largest = nums[0];
    for (double n : nums)
        if (n > largest) largest = n;
    return largest;
}
```

 Another green bullet point states: "Call the method like this:". Below this, a code block shows two calls:

```
double d1 = max(10, 20); // 2 arguments
double d2 = max(10, 20, 30, 50, 70); // 5 arguments
```

Slide 13

Variable-Length Parameters

Duration: 00:01:44

Advance mode: Auto

Notes:

When you define a method, you have to specify how many parameters it requires. That's part of the method signature. Some methods, though, like `printf()` or `String.format()`, can take a variable number of parameters. Here's how they do it.

A variable-length formal parameter is really a way of defining a special kind of array parameter. When defining the formal parameter, use an ellipsis (three dots) instead of the square brackets you'd use for a normal array declaration. Here's an example using the `max()` method, so you could pass a variable number of values.

Inside the `max()` method, you treat `nums` as a regular array of `double`. That means that you can use the standard array-processing algorithm for finding the largest value, and then return that, as shown here.

The big difference between a variable-length parameter method, and one that takes a regular array as a formal parameter, is that, for the variable-length variety, Java will create the array for you; with the "regular" version, it's up to the caller to create the array before calling the method.

That means you can call the method in either of these two ways.

You can combine a variable-length parameter with regular parameters, but the variable-length parameter must come last. You can't have more than one variable-length parameter per method.